

Guia HCWBP

Seccion Proveedores - Sistema Empresa

Aprender primero el sistema completo, despues los nombres y detalles tecnicos.

Protocolo	Como se aplica en esta guia
Big Picture	Primero entiendes que problema resuelve proveedores.
Relaciones	Luego ves como proveedores sostiene a compras.
Detalles	Solo despues entran clases, metodos, SQL y validaciones.
Practica	Termina con preguntas conceptuales y ejercicios progresivos.

1. Big Picture - Que problema resuelve

El modulo de proveedores existe para resolver una pregunta basica del negocio: a quien le compra la empresa y si esa persona o empresa esta autorizada para seguir vendiendonos.

Sin este modulo, una compra podria registrarse con datos incompletos, con un proveedor duplicado o con alguien que ya no deberia estar activo.

Problema sin proveedores	Solucion que da tu seccion
No saber a quien se le compra	Guarda nombre, RFC, telefono, correo y direccion.
Duplicar proveedores	No permite repetir RFC.
Comprar a alguien bloqueado	Usa activo = false para impedir nuevas compras.
Perder historial	Desactiva en lugar de borrar.

2. System Context - Donde encaja

Dentro del proyecto, Proveedores vive en el paquete de compras. No esta separado por casualidad: una compra necesita saber a que proveedor pertenece.

Mapa de contexto:

Usuario -> App -> ProveedorController -> ProveedorService -> ProveedorRepository -> MySQL

Y cuando se registra una compra, el sistema consulta la tabla Proveedor para confirmar que el idProveedor existe y esta activo.

3. Architecture - Componentes y relacion

Capa	Archivo	Responsabilidad
Modelo	Proveedor.java	Es el molde: guarda los datos del proveedor.
Controller	ProveedorController.java	Recibe la accion desde App y llama al Service.
Service	ProveedorService.java	Decide si la operacion se permite segun reglas de negocio.
Repository	ProveedorRepository.java	Habla con MySQL usando SQL y Conexion.

La idea importante: cada capa tiene una responsabilidad. Por eso el codigo se puede explicar sin mezclar consola, reglas y SQL.

4. Constants vs Variables

Lo que se mantiene igual	Lo que puede cambiar
La ruta por capas: Controller -> Service -> Repository.	Los datos capturados: nombre, RFC, telefono, correo, direccion.
El proveedor se guarda con activo = true al registrarse.	El idProveedor cambia porque lo genera la base de datos.
El RFC debe ser unico.	El estado activo puede cambiar a false al desactivar.

Lo que se mantiene igual	Lo que puede cambiar
Las compras revisan que el proveedor este activo.	La lista de proveedores crece con cada registro nuevo.

5. Mental Model - Analogia simple

Piensa en Proveedores como la libreta oficial de contactos autorizados de una empresa.

- Proveedor.java es la ficha de contacto.
- ProveedorController es quien recibe la solicitud de usar la libreta.
- ProveedorService es quien revisa si la ficha esta completa y si no esta repetida.
- ProveedorRepository es quien guarda o busca la ficha en el archivero, que en este caso es MySQL.
- activo = false es como poner una marca de 'no comprarle mas', sin romper la ficha historica.

6. Visualize the Flow - Input -> Process -> Output

Operacion	Input	Process	Output
Registrar	Datos del proveedor	Validar campos, correo, RFC unico y guardar	Proveedor activo en MySQL
Mostrar	Ninguno	Consultar todos los registros	Lista de proveedores
Buscar	idProveedor	Consultar por id y validar existencia	Proveedor encontrado o error
Desactivar	idProveedor	Buscar, validar existencia y actualizar activo	Proveedor inactivo

Flujo especifico de registrar:

- Input: el usuario escribe nombre, RFC, telefono, correo y direccion.
- Process: el Service revisa que los datos tengan sentido.
- Process: el Repository consulta si el RFC ya existe.
- Output: si todo esta bien, se guarda el proveedor activo.

7. Ahora si: detalles tecnicos

Concepto tecnico	Significado despues de entender el sistema
Clase	Un bloque de codigo con una responsabilidad concreta.
Objeto Proveedor	Una ficha individual de proveedor con sus datos.
Metodo	Una accion que una clase sabe hacer.
PreparedStatement	Forma segura y ordenada de enviar valores a una consulta SQL.
ResultSet	Resultado que devuelve MySQL despues de una consulta SELECT.
RuntimeException	Error que sube a App para mostrar un mensaje sin cerrar todo el programa.

SQL principal de relacion con compras:

```
SELECT COUNT(*) FROM Proveedor WHERE id_proveedor = ? AND activo = ?
```

Esta consulta contesta: existe este proveedor y todavía está permitido comprarle?

8. WHY - Por que existen las reglas

Regla	Por que existe	Que pasaria sin ella
Nombre obligatorio	Identifica a quien se compra.	Habria proveedores imposibles de reconocer.
RFC obligatorio y unico	Evita duplicidad fiscal.	Dos proveedores podrian parecer diferentes siendo el mismo.
Telefono obligatorio	Permite contacto operativo.	No habria forma directa de resolver problemas de compra.
Correo con @	Filtra correos claramente invalidos.	Se guardarian contactos inutilles.
Desactivar, no borrar	Conserva historial.	Compras pasadas podrian perder referencia.
Compra solo con activo = true	Bloquea nuevas operaciones con proveedores inactivos.	Se podria comprar a alguien dado de baja.

9. Check My Understanding

Antes de memorizar metodos, responde estas preguntas en voz alta:

- Por que Proveedores es necesario antes de registrar una compra?
- Que capa decide si un RFC repetido se acepta o se rechaza?
- Por que Repository si conoce SQL, pero Controller no?
- Que significa desactivar un proveedor en lugar de eliminarlo?
- Que podria salir mal si Compra no revisara activo = true?

10. Practice - De facil a dificil

Nivel	Ejercicio
1	Explica en una frase que hace Proveedor.java.
2	Dibuja la ruta App -> Controller -> Service -> Repository -> MySQL.
3	Explica que pasa cuando intentas registrar un RFC repetido.
4	Defiende por que se usa activo = false en vez de borrar.
5	Explica como tu modulo protege al modulo de compras.
6	Simula una pregunta del profesor: 'Por que separaron Service y Repository?' y respondela sin hablar de sintaxis primero.

11. Guion final con tu estilo

Mi seccion resuelve el control de a quienes puede comprarles la empresa. Primero registra proveedores con datos minimos confiables. Luego protege al modulo de compras porque una compra solo puede hacerse con un proveedor existente y activo. La arquitectura esta separada por capas: el modelo representa datos, el controller recibe acciones, el service valida reglas y el repository consulta MySQL. La decision mas importante es desactivar en vez de borrar, porque asi se bloquean nuevas compras sin perder historial.